```
    sfs_file_entry_t fe;

    if ((i = sfs_lookup(sfs_handle, fn, &fe)) == -1) {
        printf("File %s doesn't exist\n", fn);
        return;
    }
    cur_read_i = 0;
    to_read = sb.block_size;
    total_size = 0;
    block_i = 0;
    while ((cur_read = read(0, block + cur_read_i, to_read)) >
0) {
        if (cur_read == to_read) {
            /* Write this block */
            if (block_i == SIMULA_FS_DATA_BLOCK_CNT)
                break; /* File size limit */
            if ((free_i = get_data_block(sfs_handle)) == -1)
                break; /* FS full */
            lseek(sfs_handle, free_i * sb.block_size, SEEK_SET);
            write(sfs_handle, block, sb.block_size);
            fe.blocks[block_i] = free_i;
            block_i++;
            total_size += sb.block_size;
            /* Reset various variables */
            cur_read_i = 0;
            to_read = sb.block_size;
        }
        else {
            cur_read_i += cur_read;
            to_read -= cur_read;
        }
    }
    if ((cur_read <= 0) && (cur_read_i)) {
        /* Write this partial block */
        if ((block_i != SIMULA_FS_DATA_BLOCK_CNT) &&
            ((fe.blocks[block_i] = get_data_block(sfs_handle))
!= -1)) {
            lseek(sfs_handle, fe.blocks[block_i] * sb.block_size,
SEEK_SET);
            write(sfs_handle, block, cur_read_i);
            total_size += cur_read_i;
        }
    }

    fe.size = total_size;
    fe.timestamp = time(NULL);
    lseek(sfs_handle, sb.entry_table_block_start * sb.block_size
+ i * sb.entry_size, SEEK_SET);
    write(sfs_handle, &fe, sizeof(sfs_file_entry_t));
}
```

The last stride

With the above three Simula FS command functions, the final change to the *browse_sfs()* function in *browse_sfs.c* (refer to the previous article) would be to add the cases for handling these three new commands: *chperm*, *write*, and *read*.

One of the daunting questions is how do you find free, available blocks? Notice that in *sfs_write()*, you just called a function *get_data_block()*, and everything went smoothly. But think through how that would be implemented. Do you need to traverse all the file entries every time to figure out which has been used, and determine that the remaining are free? That would be killing. Instead, an easier technique would be to get the used ones by parsing all the file entries, only initially once, and then keep track of whenever more entries are used or freed up. But for that, a complete framework needs to be put in place, which includes the following:

- *uint1_t typedef (in sfs_ds.h)*
- *used_blocks* dynamic array to keep track of used blocks (in *browse_sfs.c*)
- Function *init_browsing()* to initialise the dynamic array *used_blocks*, i.e., allocate and mark the initial used blocks (in *browse_sfs.c*)
- Correspondingly, the inverse function *shut_browsing()* to clean up the same (in *browse_sfs.c*)
- And, definitely, the functions *get_data_block()* and *put_data_block()* to get and put back, respectively, the free data blocks based on the dynamic array *used_blocks* (in *browse_sfs.c*)

All these thoughts, incorporated with the earlier *sfs_ds.h* and *browse_sfs.c* files, are available from *http://linuxforu.com/article_source_code/july12/sfs_code.tar.bz2*. Once compiled and executed as *./sfs_browse*, it shows up as something like what is in Figure 1.

As Shweta showed her working demo to her project mate, he observed some issues that she had missed out, and challenged her to find them out on her own. He hinted that they were related to the newly added functionality and 'getting a free block' framework. He added that some were even visible from the demo, i.e., Figure 1. Can you help Shweta find them out? If yes, send your answers to *shweta@sarika-pugs.com*. END

By: Anil Kumar Pugalia

The author is a freelance trainer in Linux internals, Linux device drivers, embedded Linux and related topics. Prior to this, he was at Intel and Nvidia. He has been working with Linux since 1994. A gold medallist from the Indian Institute of Science, Linux and knowledge-sharing are two of his many passions. Creating and playing with open source hardware is one of his hobbies. Learn more about him at *http://profession.sarika-pugs.com/*, including information about his open source hardware freak-outs, and his various Linux related training sessions and workshops. He can be reached at *email@sarika-pugs.com*.